

Slovenská technická univerzita v Bratislave
Fakulta informatiky a informačných technológií

Princípy softvérového inžinierstva

Akademický rok 2025/26

Dokumentácia implementácie systému BikeFlow

Fáza 5

Autori:

Andrej Mikloš
Mykhailo Chepara
Dmytro Petrenko
Marharyta Kalyna

Bratislava, 2026

Obsah

Obsah.....	3
1. Úvod	4
2. Technologický základ	4
3. Mapovanie na architektúru F4	4
3.1 Tabuľka mapovania tried.....	4
3.2 BCE architektonické vrstvy	5
Boundary vrstva (Hraničné triedy)	5
Control vrstva (Riadiace triedy)	5
Entity vrstva (Doménové triedy)	5
4. Mapovanie na prípady použitia	6
UC01: Rezervácia servisného termínu zákazníkom	6
UC02: Spracovanie a riadenie servisnej opravy bicykla	6
UC03: Zadáť požiadavku na expedíciu tovaru (vrátane UC03a, UC03b)	6
UC04: Prenajmi si bicykel	6
UC05: Predbežne registruj.....	7
UC06: Spracuj neprítomnosť alebo nesprávnosť bicykla	7
5. Nové triedy pridané v 5. Faze Odovzdania	7
5.1 Rozhranie Cancellable (common/Cancellable.java)	7
5.2 Rozhranie Notifiable (common/Notifiable.java)	7
5.3 Entita Feedback (rental/domain/Feedback.java).....	7
5.4 Doménové triedy ServiceBooking (servicebooking/domain/)	8
5.5 Spare parts (sparepart/domain/)	8
5.6 WarehouseStaff (dispatch/domain/WarehouseStaff.java).....	8
5.7 Hodnotové objekty Common balíka	8
6. Databázová schéma	8
7. Spustenie a testovanie	8
Požiadavky	8
Spustenie aplikácie	8
Spustenie testov	8
8. Záver	9

1. Úvod

Táto dokumentácia popisuje implementáciu systému BikeFlow v jazyku Java s využitím frameworku Spring Boot 3.5. Kód implementuje vybrané prípady použitia (UC01–UC06) definované v predchádzajúcich fázach projektu a rešpektuje architektonický model Boundary–Control–Entity (BCE) navrhnutý vo Fáze 4.

Kód je spustiteľný v prostredí IntelliJ IDEA. Na spustenie je potrebné mať nainštalovanú Javu 21 a Maven. Aplikácia používa vstavanú H2 databázu, takže nie sú potrebné žiadne externé závislosti.

2. Technologický základ

- Jazyk: Java 21
- Framework: Spring Boot 3.5.0
- Persistencia: Spring Data JPA + H2 (in-memory databáza)
- Migrácie: Flyway (4 migračné skripty + seed dáta)
- Testovanie: JUnit 5, Mockito
- Build: Maven

Štruktúra projektu dodržiava konvencie Spring Boot a je organizovaná podľa doménových balíkov, čo zodpovedá architektúre definovanej v BCE modeli.

3. Mapovanie na architektúru F4

Implementácia vychádza z diagramov tried navrhnutých vo Fáze 4. Nasledujúca tabuľka zobrazuje mapovanie medzi F4 triedami a ich Spring Boot ekvivalentmi:

3.1 Tabuľka mapovania tried

F4 trieda (diagram tried)	Implementovaná trieda / balík	Poznámka
RentalReservation	rental/domain/Rental.java	Premenovaná podľa Spring Boot konvencií
Feedback	rental/domain/Feedback.java	Nová entita pridaná v F5
ServiceReservation	servicebooking/domain/ServiceBooking.java	Zlúčená s ServiceOrder pre jednoduchosť
ExpeditionRequest	dispatch/domain/ExpeditionRequest.java	Zodpovedá F4 ExpeditionRequest
RequestItem	dispatch/domain/ExpeditionRequestItem.java	Zodpovedá F4 RequestItem
StockItem	inventory/domain/StockItem.java	Rozšírená o minimálne množstvo, isBelowMinimum()

F4 trieda (diagram tried)	Implementovaná trieda / balík	Poznámka
SaleRecord	inventory/domain/SaleRecord.java	Zodpovedá F4 SaleRecord
Facility	facility/domain/Facility.java	Priame mapovanie
Product	product/domain/Product.java	Priame mapovanie
Bike	bike/domain/Bike.java	Priame mapovanie, Common balík
Cancellable	common/Cancellable.java	Interface implementovaný Rental a ServiceBooking
Notifiable	common/Notifiable.java	Interface implementovaný Rental a ServiceBooking
BacklogicController	rental/application/RentalService.java	Control vrstva pre prenájom
ReservationController	servicebooking/application/ServiceBookingService.java	Control vrstva pre servis
DistributionController	dispatch/application/ExpeditionRequestService.java	Control vrstva pre distribúciu
WebApp, ReservationFormUI, ...	static/index.html + app.js	Boundary vrstva (HTML/JS UI)

3.2 BCE architektonické vrstvy

Implementácia dôsledne dodržiava trojvrstvový BCE model:

Boundary vrstva (Hraničné triedy)

Všetky REST controllery (RentalController, ServiceBookingController, ExpeditionRequestController, atď.) a HTML/JavaScript frontend (index.html, app.js) tvoria hraničnú vrstvu. Tieto triedy prijímajú vstupy od používateľov a delegujú ich na Control vrstvu.

Control vrstva (Riadiace triedy)

Spring @Service triedy (RentalService, ServiceBookingService, ExpeditionRequestService) koordinujú biznis logiku. Zodpovedajú kontrolérom z F4 diagramov (BacklogicController, ReservationController, DistributionController).

Entity vrstva (Doménové triedy)

JPA entity v balíkoch domain/ (Rental, ServiceBooking, ExpeditionRequest, Bike, Facility, atď.) reprezentujú doménové objekty. Nemajú žiadne vedomie o Control ani Boundary vrstvách.

Závislosť medzi vrstvami je jednosmerná: Boundary → Control → Entity, čo priamo zodpovedá F4 diagramu balíkov.

4. Mapovanie na prípady použitia

UC01: Rezervácia servisného termínu zákazníkom

Implementované triedy:

- ServiceBookingController.java (Boundary) – REST endpoint POST /api/service-bookings
- ServiceBookingService.create() (Control) – automaticky vyhľadá prvý voľný slot v požadovanom časovom okne; overuje kapacitu servisu (max. 2 súčasné rezervácie na slot)
- ServiceBooking.java (Entity) – uchováva stav rezervácie

Kľúčová metóda: ServiceBookingService.create() – algoritmus vyhľadávania voľného termínu realizuje kroky 4–6 hlavného scenára UC01.

UC02: Spracovanie a riadenie servisnej opravy bicykla

Implementované triedy:

- ServiceBookingController.java (Boundary) – REST endpoint PATCH /api/service-bookings/{id}/status
- ServiceBookingService.updateStatus() (Control) – aktualizuje stav objednávky, nastavuje predbežnú cenu a termín dokončenia
- ServiceBookingStatus.java – stavy: SCHEDULED, RECEIVED, WAITING_FOR_PARTS, IN_REPAIR, DONE, CANCELLED, REJECTED, NO_SHOW

Kľúčová metóda: ServiceBookingService.updateStatus() realizuje kroky 9–11 hlavného scenára UC02 (zaznamenanie dokončenia, výpočet ceny).

UC03: Zadať požiadavku na expedíciu tovaru (vrátane UC03a, UC03b)

Implementované triedy:

- ExpeditionRequestController.java (Boundary) – REST endpointy GET /api/expedition-requests, POST /api/expedition-requests
- ExpeditionRequestService.create() (Control) – validuje dostupnosť zásob v zdrojovej prevádzke (UC03b), overuje, že zdrojová a cieľová prevádzka sú rôzne
- ExpeditionRequest.java, ExpeditionRequestItem.java (Entity) – F4 triedy ExpeditionRequest a RequestItem
- InventoryController.getSalesAnalysis() (Boundary) – implementuje UC03a (analýza predaja)

Kľúčová metóda: ExpeditionRequestService.create() realizuje kroky 9–17 hlavného scenára UC03, vrátane kontroly dostupnosti (UC03b).

UC04: Prenajmi si bicykel

Implementované triedy:

- RentalController.java (Boundary) – endpointy POST /start, POST /finish
- RentalService.startRental(), RentalService.finishRental() (Control) – riadia životný cyklus prenájmu: PRELIMINARY → ACTIVE → COMPLETED
- Rental.java (Entity) – implementuje Cancellable a Notifiable

Kľúčová metóda: RentalService.finishRental() realizuje kroky 10–11 UC04 (výpočet ceny, odpočítanie kreditov, uvoľnenie bicykla).

UC05: Predbežne registruj

Implementované triedy:

- RentalController.preRegister() (Boundary) – endpoint POST /api/rentals/pre-register
- RentalService.preRegister() (Control) – overuje aktivnosť zákazníka, verifikovanú platobnú kartu a dostatočný kredit; nastavuje stav bicykla na PRE_RESERVED

Kľúčová metóda: RentalService.preRegister() realizuje kroky 3–8 hlavného scenára UC05.

UC06: Spracuj neprítomnosť alebo nesprávnosť bicykla

Implementované triedy:

- RentalController.reportIssue() (Boundary) – endpoint POST /api/rentals/{id}/issue
- RentalService.reportIssue() (Control) – označí bicykel ako DAMAGED alebo MISSING, zruší rezerváciu, vytvorí záznam RentalIssueReport
- RentalIssueReport.java (Entity) – evidencia hlásenia

Kľúčová metóda: RentalService.reportIssue() realizuje kroky 3–4 hlavného scenára UC06.

5. Nové triedy pridané v 5. Faze Odovzdania

5.1 Rozhranie Cancellable (common/Cancellable.java)

Formalizuje kontrakt pre doménové objekty, ktorých životný cyklus môže byť explicitne ukončený zrušením. Zodpovedá rozhraniu Cancellable z F4 Common balíka. Implementované triedami Rental a ServiceBooking.

Metódy:

- cancel() – nastaví interný stav na CANCELLED
- isCancelled() – overí, či je objekt zrušený

5.2 Rozhranie Notifiable (common/Notifiable.java)

Formalizuje kontrakt pre doménové objekty, ktoré nesú kontaktné informácie potrebné na doručenie notifikácií. Zodpovedá rozhraniu Notifiable z F4 Common balíka. Implementované triedami Rental (email zákazníka) a ServiceBooking (email zákazníka).

Metódy:

- getEmail() – vracia email príjemcu notifikácie
- getPhone() – vracia referenčné číslo (napr. číslo prenájmu alebo rezervácie)

5.3 Entita Feedback (rental/domain/Feedback.java)

Záznam spokojnosti zákazníka po dokončení prenájmu. Zodpovedá triede Feedback z F4 Rental Class Diagramu. Zákazník môže zanechať hodnotenie (1–5) a komentár po skončení prenájmu (UC04, krok 12).

Atribúty:

- id – UUID primárny kľúč
- rental – väzba 1:1 na Rental (jeden prenájom, maximálne jedno hodnotenie)
- rating – hodnotenie 1–5
- comment – voliteľný textový komentár
- submittedAt – časová pečiatka odoslania

Pridaný endpoint: POST /api/rentals/{rentalId}/feedback

5.4 Doménové triedy ServiceBooking (servicebooking/domain/)

- TimeSlot – obaľuje OffsetDateTime; metódy next(), isAfter()
- ServiceCapacity – uchováva maxSlots; metóda isFull(activeCount)
- ServiceWorker – entita; atribúty fullName, email; väzba na Facility (servicePoint)

5.5 Spare parts (sparepart/domain/)

- SparePart – entita; atribúty sku, name, stockQuantity
- OrderSparePart – väzobná entita ServiceBooking ↔ SparePart + quantity
- PartOrder – externá objednávka dielov; status ORDERED/DELIVERED

Pridaný endpoint: POST /api/service-bookings/{id}/parts

5.6 WarehouseStaff (dispatch/domain/WarehouseStaff.java)

Zamestnanec skladu oprávnený vytvárať expedičné požiadavky. Atribúty: fullName, email, väzba na Facility (warehouse). Väzba na ExpeditionRequest cez pole requestedBy.

5.7 Hodnotové objekty Common balíka

- DateRange (@Embeddable) – from/to + contains(); použitý v ServiceBooking.preferredWindow
- NotificationService – cross-cutting služba; zasiela potvrdenia a stavové notifikácie cez SLF4J log

6. Databázová schéma

Schéma je vytváraná automaticky cez Hibernate (ddl-auto=create-drop). Flyway je vypnuté (spring.flyway.enabled=false). Aplikácia používa vstavanú H2 in-memory databázu – PostgreSQL nie je potrebný. Seed dáta sú načítané zo súboru data.sql pri každom štarte (spring.sql.init.mode=always). H2 konzola je dostupná na /h2-console (JDBC URL: jdbc:h2:mem:bikeflow, user: sa).

7. Spustenie a testovanie

Požiadavky

- Java 21
- Maven 3.9+ (alebo IntelliJ IDEA 2025 – spustenie cez Run Configuration)
- PostgreSQL nie je potrebný – H2 in-memory databáza je súčasťou projektu

Spustenie aplikácie

Spustenie projektu sa vykoná príkazom:

```
mvn spring-boot:run
```

Aplikácia beží na http://localhost:8080. Webové rozhranie pre manuálne testovanie všetkých UC je dostupné priamo na tejto adrese.

Spustenie testov

Unit testy sa spustia príkazom:


```
mvn test
```

Triedy `RentalServiceTest` a `ServiceBookingServiceTest` overujú kľúčovú biznis logiku UC01–UC06 pomocou Mockito mockov. `RentalServiceTest` pokrýva pre-registráciu, zrušenie prenajmu (bez refundu kreditu) a chybný stav. `ServiceBookingServiceTest` pokrýva vytvorenie rezervácie a spracovanie opravy s výpočtom ceny a loyalty zľavou.

8. Záver

Implementácia pokrýva šesť hlavných prípadov použitia (UC01–UC06) definovaných v predchádzajúcich fázach. Kód je organizovaný podľa BCE architektonického vzoru s jasným oddelením Boundary (REST controllery + UI), Control (Spring `@Service` triedy) a Entity (JPA doménové triedy) vrstiev. Nové triedy pridané v tejto fáze (`Cancellable`, `Notifiable`, `Feedback`) priamo zodpovedajú triedam navrhnutým v F4 class diagramoch, čím sa uzatvára konzistencia medzi návrhovým modelom a implementáciou.